Secure Coding Review Report

Task 3: Secure Coding Review

1. Introduction

Secure coding is the practice of developing software that is resistant to security vulnerabilities and attacks. The objective of this review is to identify security weaknesses in a Python Flask web application and provide recommendations to improve its security.

Selected Programming Language

Python

Selected Application

Flask-based User Registration Application

The application allows users to register using a username and password. A code review was performed using manual inspection techniques and static analysis principles.

---

2. Methodology

The security audit was conducted using the following methods:

Manual Code Review

The source code was examined line by line to identify insecure coding practices and potential vulnerabilities.

Static Analysis

The Bandit security analyzer was used to scan the Python source code for common security issues and coding weaknesses.

Testing Environment

- Operating System: Windows
- IDE: Visual Studio Code
- Programming Language: Python

- Framework: Flask
- Static Analysis Tool: Bandit

---

3. Source Code Under Review

The application contains a registration feature that stores user information and runs a Flask web server.

The review focused on:

- Credential management
- Password handling
- Input validation
- Application configuration
- Authentication controls

---

4. Security Findings

Finding 1: Hardcoded Credentials

Description

Administrative credentials were directly embedded in the source code.

Example:

```
admin_username = "admin"
admin_password = "123456"
```

Risk

Hardcoded credentials can be exposed through source code leaks, unauthorized repository access, or reverse engineering.

Severity

High

Recommendation

Store credentials in environment variables or a secure secrets management system.

Finding 2: Plain Text Password Storage

Description

User passwords were stored without encryption or hashing.

Example:

"password": password

Risk

If the application data is compromised, attackers can immediately access user passwords.

Severity

High

Recommendation

Use secure password hashing algorithms such as bcrypt or Werkzeug password hashing functions before storing passwords.

Finding 3: Missing Input Validation

Description

The application accepts user input without validation.

Example:

username = request.form['username']

Risk

Malicious input may lead to application errors, injection attacks, or unexpected behavior.

Severity

Medium

Recommendation

Validate and sanitize all user inputs before processing.

Finding 4: Debug Mode Enabled

Description

The Flask application runs with debug mode enabled.

Example:

app.run(debug=True)

Risk

Debug mode may reveal sensitive information such as stack traces, source code details, and server configuration.

Severity

Medium

Recommendation

Disable debug mode in production environments.

Finding 5: Lack of Authentication Controls

Description

The application does not implement proper authentication or authorization mechanisms.

Risk

Unauthorized users may access application functionality without restrictions.

Severity

Medium

Recommendation

Implement user authentication, session management, and access control mechanisms.

---

5. Remediation Steps

To improve application security, the following actions should be implemented:

1. Remove hardcoded credentials.
2. Use environment variables for secrets.
3. Hash passwords before storage.
4. Validate and sanitize all user inputs.
5. Disable debug mode in production.
6. Implement authentication and authorization controls.
7. Perform regular security testing.
8. Keep dependencies updated.
9. Apply secure coding standards.
10. Conduct periodic code reviews.

---

6. Best Practices for Secure Coding

- Follow the Principle of Least Privilege.
- Protect sensitive information.
- Use secure password storage methods.
- Validate all external input.
- Use HTTPS for communication.
- Log security events appropriately.
- Regularly update software components.
- Conduct security assessments throughout development.

---

7. Conclusion

The secure coding review identified several vulnerabilities within the Python Flask application. The most critical issues were hardcoded credentials and plain text password storage. Additional weaknesses included missing input validation, enabled debug mode, and insufficient authentication controls.

By implementing the recommended remediation steps and secure coding best practices, the application's security posture can be significantly improved. Regular security reviews and automated scanning tools should be incorporated into the software development lifecycle to ensure long-term protection against emerging threats.

References

1. OWASP Top 10 Web Application Security Risks.
2. Python Security Best Practices Documentation.
3. Flask Official Documentation.
4. Bandit Security Analyzer Documentation.
5. NIST Secure Software Development Framework (SSDF).